

P1467R8

Extended floating-point types and standard names

Published Proposal, 2021-12-16

This version:

<https://wg21.link/P1467R8>

Authors:

[David Olsen](#) (NVIDIA)

[Ilya Burylov](#) (Intel)

[Michał Dominiak](#) (NVIDIA)

Audience:

EWG, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1	Abstract
2	Revision history
2.1	R0 -> R1 (pre-Cologne)
2.2	R1 -> R2 (pre-Belfast)
2.3	R2 -> R3 (pre-Prague)
2.4	R3 -> R4 (Summer 2020)
2.5	R4 -> R5 (Fall 2021)
2.6	R5 -> R6 (Fall 2021)
2.7	R6 -> R7 (Fall 2021)

2.8 R7 -> R8 (December 2021)

3 Motivation

4 C Compatibility

5 Core language changes

5.1 Things that aren't changing

5.2 Extended floating-point types

5.2.1 Reasoning

5.3 Conversion rank

5.3.1 Reasoning

5.4 Promotion

5.5 Implicit conversions

5.5.1 Reasoning

5.5.2 Constant values

5.5.2.1 Standard conversion of constants

5.5.2.2 Direct initialization with constants

5.6 Usual arithmetic conversions

5.6.1 Example

5.7 Narrowing conversions

5.7.1 Interaction with implicit conversions

5.8 Overload resolution

5.8.1 Examples

5.8.2 Alternate proposals

5.8.2.1 Prefer smallest safe conversions

5.8.2.2 No change

5.8.3 Comparisons

5.9 Pointer conversions

5.10 Literal suffixes

5.11 Feature test macro

6 Library changes

6.1 Possible new names

6.1.1 Standard/extended floating-point traits

6.1.2 Conversion rank trait

- 6.2 `<charconv>`
- 6.3 `<format>`
- 6.4 I/O Streams
- 6.5 `<cmath>`
- 6.6 `<complex>`
- 6.7 `<atomic>`
- 6.8 Feature test macro

7 **Type aliases**

- 7.1 Header name
- 7.2 Supported formats
- 7.3 Names
 - 7.3.1 C compatibility
- 7.4 Literal suffixes
- 7.5 Aliasing standard types
- 7.6 Layout vs. behavior
- 7.7 Feature test macros

8 **Implementation experience**

9 **Wording**

- 9.1 References
- 9.2 Core
 - 9.2.1 Literal suffixes
 - 9.2.2 Extended floating-point types
 - 9.2.3 Conversion rank
 - 9.2.4 Implicit conversions
 - 9.2.5 Usual arithmetic conversions
 - 9.2.6 Narrowing conversions
 - 9.2.7 Overload resolution
- 9.3 Library
 - 9.3.1 `<stdfloat>`
 - 9.3.2 `<charconv>`
 - 9.3.3 I/O Streams
 - 9.3.3.1 `<ostream>`

9.3.3.2	<code><istream></code>
9.3.4	<code><cmath></code>
9.3.5	<code><complex></code>
9.3.6	<code><atomic></code>
9.3.7	Feature test macros

References

Informative References

§ 1. Abstract

Allow implementations to define *extended floating-point types* in addition to the three standard floating-point types. Define rules for how the extended floating-point types interact with each other and with other types without changing the behavior of the existing standard floating-point types. Specify the rules for type conversions, arithmetic conversions, narrowing conversions, and overload resolution in a way that strikes a balance between behaving like existing types and encouraging safe code. Specify the necessary library support, mostly additional overloads for functions that take floating-point arguments, for the extended floating-point types.

Define an optional set of `<cstdint>`-style type aliases for floating-point types matching specific, well-known floating-point layouts.

§ 2. Revision history

§ 2.1. R0 -> R1 (pre-Cologne)

Applied guidance from [SG6](#) in Kona 2019:

1. Make the floating-point conversion rank not ordered between types with overlapping (but not subsetting) ranges of finite values. This makes the ranking a partial order.
2. Narrowing conversions are now based on floating-point conversion rank instead of ranges of finite values, which preserves the current narrowing conversions relations between standard floating-point types; it also interacts favorably with the rank being a partial ordering.
3. Operations that deal with floating-point types whose conversion ranks are unordered are now ill-formed.
4. The relevant parts of the guidance have been applied to the library wording section as well.

Afterwards, applied suggestions from [EWGI](#) in Kona 2019 (this modifies some of the points above):

1. Apply the suggestion to make types where one has a wider range of finite values, but a lower precision than the other, unordered in their conversion rank, and therefore make operations that mix them ill-formed. The motivating example was IEEE-754 [binary16](#) and [bfloat16](#); see [Floating-point conversion rank](#) for more details. This change also caused this paper to drop the term "range of finite values", since the modified semantics are better expressed in terms of sets of values of the types.
2. Add a change to narrowing conversions, to only allow exact conversions to happen.
3. Explicitly list parts of the language that are not changed by this paper; provide a more detailed analysis of the standard library impact.

§ 2.2. R1 -> R2 (pre-Belfast)

Changes based on feedback in Cologne from [SG6](#), [LEWGI](#), and [EWGI](#). Further changes came from further development of the paper by the authors, especially [overload resolution](#).

- Revised floating-point [promotion](#) rules. Removed all promotions other than `float` to `double`. Added wording for promoting values passed to varargs functions.
- Added the section on [implicit conversions](#).
- Added the section on [overload resolution](#).
- Added the section about [feature test macros](#).
- Added the sections about the possibility of new [library traits](#).
- Changed the wording for the `abs` function in the `<cmath>` section.
- Added constraints to the I/O streams overloads for `complex` to only support standard floating-point types.
- Added the section about possible changes to `<atomic>`.

§ 2.3. R2 -> R3 (pre-Prage)

Changes based on feedback in Belfast from [EWG](#).

- Change the [overload resolution](#) rules, removing the rule that prefers one standard conversion over another based on conversion rank. Replace it with a rule that prefers one standard conversion over another only when the two types have the same representation.

- As a result of the overload resolution change, change floating-point [promotion](#) so that any type smaller than `double` promotes to `double`.
- Allow implicit conversions between pointer types that point to floating-point types with the same representation.

§ 2.4. R3 -> R4 (Summer 2020)

Merge [P1468](#) into P1467. The two papers were separate proposals when first written. But over time they have become intertwined, with design decisions in one paper affecting the feasibility of the other. So the two papers are being merged into a single proposal in P1467R4.

Changes based on feedback in Prague from EWG, where the discussion was all about what the goals of the proposal should be. The group settled on a set of design decisions (see the [poll results](#)) that strike a balance between the existing behavior of arithmetic types and a "safe by default" strategy.

Changes between [P1467R3](#) and P1467R4:

- Add section [§ 4 C Compatibility](#)
- Revert the rules for floating-point [§ 5.4 Promotion](#) back to what they were in [P1647R2](#), which is essentially unchanged from the current C++ standard. This was necessitated by changes to the overload resolution rules.
- Resolve the open issue of [§ 5.5 Implicit conversions](#). In R3, it was undecided if potentially lossy conversions should be implicit. EWG in Prague was strongly in favor of requiring lossy conversions to be explicit. The section on implicit conversions now reflects that guidance.
- Revert the rules for [§ 5.8 Overload resolution](#) back to what they were in [P1647R2](#), with a small fix to the proposed wording changes. Two alternate ideas for overload resolution are now listed.
- Withdraw the proposed change for [§ 5.9 Pointer conversions](#).

Changes to the content of [P1468R3](#) as it was merged into P1467R4:

- Changed the proposed [§ 7.4 Literal suffixes](#) to match what will be available in C2x.

§ 2.5. R4 -> R5 (Fall 2021)

Rebase wording to C++20.

Separate the design and wording sections, with links between them.

Improve the section on C Compatibility, adding more discussion about the use of different names in the two languages and a section about differences in usual arithmetic conversions.

Remove the part of the proposal that promoted types smaller than `double` to `double` when passed to varargs functions.

Add more explanation to the section about overload resolution.

Fill in the section about `<format>`.

Add support for I/O Streams of extended floating-point types that are no larger than `long double`.

Add background information for the sections on `<charconv>` and `<cmath>`.

Decide on one set of names, `std::floatN_t`, for the type aliases of types with well-known formats.

§ 2.6. R5 -> R6 (Fall 2021)

Based on discussions on SG22 and EWG mailing lists and an SG22/CFP teleconference, make a slight change to usual arithmetic conversions to match C23's behavior. The best way to do that is to change the definition of conversion rank, splitting it into rank and subrank. This leads to very slight changes to implicit conversions and narrowing conversions. The description and wording for all these sections has changed, though the changes that would be noticed by a programmer are very minor. The change to conversion rank also results in changes to the wording for some library features, though no change in behavior.

Change the overload resolution section significantly, switching the proposal from "prefer smallest safe conversion" to "prefer same conversion rank."

No longer propose adding any new type traits. The discussion is still in the paper, but the recommendation is for no change. See [§ 6.1 Possible new names](#)

Choose `<stdfloat>` as the name of the header for the new type aliases.

Add a paragraph to [§ 7.3.1 C compatibility](#) discussing the implications of C23 names `_FloatN_t`.

Request polls from LEWG about whether or not the `_FloatN` names should be required to be available in C++, and about whether the literal suffixes should be a language feature or a library feature.

Add preliminary wording for the type aliases and their literal suffixes.

§ 2.7. R6 -> R7 (Fall 2021)

Based on discussions and polls in an [LEWG teleconference](#): Change the literal suffixes from a library feature to a language feature. Rewrite the sections on feature test macros, adding a library feature test macro to the proposal. Settle on not requiring that the C names (`_FloatN`) be available in C++.

Add wording to move the reference to the IEEE/IEC floating-point standard from the bibliography to the normative references section.

Reorder the subsections in [§ 7 Type aliases](#) to be in a more logical order.

Rebased the wording onto N4901. (Except for one paragraph in [basic.fundamental], the only changes were section numbers.)

§ 2.8. R7 -> R8 (December 2021)

Fix the wording in [basic.fundamental] to deal with cv-qualified types now being counted among the floating-point types. Add "cv-unqualified" in several other places in the wording to exclude those cv-qualified types from being included in template specializations or overload sets.

Add small wording changes to [expr.spaceship] and [c.math.fpclass] to implement the proposed design in places that we had missed previously.

Add a new section about [implicit conversions and constant values](#), which discusses a complication with the implicit conversion rules.

Add a new section about the interaction between [implicit conversions and narrowing conversions](#).

Add more examples for overload resolution. Change the wording for overload resolution to also cover choosing between FP2 -> FP1 and FP3 -> FP1.

Add a section about (the lack of) implementation experience.

§ 3. Motivation

16-bit floating-point support is becoming more widely available in both hardware (ARM CPUs, NVIDIA GPUs, and, as of recently, Intel CPUs) and software (OpenGL, CUDA, and LLVM IR). Programmers wanting to take advantage of 16-bit floating-point support have been stymied by the lack of built-in compiler support for the type. A common workaround is to define a class type with all of the conversion operators and overloaded arithmetic operators to make it behave as much as possible

like a built-in type. But that approach is cumbersome and incomplete, requiring inline assembly or other compiler-specific magic to generate efficient code.

The problem of efficiently using newer floating-point types that haven't traditionally been supported can't be solved through user-defined libraries. A possible solution of an implementation changing `float` to be a 16-bit type would be unpopular because users want support for newer floating-point types in addition to the standard types, and because users have come to expect `float` and `double` to be 32- and 64-bit types and have lots of existing code written with that assumption.

This problem is worth solving, and there is no viable solution under the current standard. So changing the core language in an extensible and backward-compatible way is appropriate. Providing a standard way for implementations to support 16-bit floating-point types will result in better code, more portable code, and wider use of those types.

While deciding what names to give to the 16-bit floating-point types, it was decided that C++ would benefit from having standard names for other larger floating-point types that are commonly used. Having names for specific floating-point formats allows users to more clearly specify their intent. If a user writes code that is designed for an IEEE 64-bit binary floating-point type, the code is more clear if it uses a name that is guaranteed to be IEEE 64-bit, and the failure mode is more immediate (a compilation error) if the code is ported to a system where an IEEE 64-bit type is not available. This part of the proposal is a revival, with major modifications, of [\[N1703\]](#), which in 2013 proposed adding typedefs for fixed-layout floating-point types to both C and C++, but was not adopted by either language.

The motivation for the current approach of extended floating-point types comes from discussion of the previous paper [\[P0192\]](#). That proposal's single new standard type of `short float` was considered insufficient, preventing the use of both IEEE-754 16-bit and `bfloat16` in the same application. When that proposal was rejected in November 2018, the current, more expansive, proposal was developed. It is not feasible to predict which floating-point types, or even how many different types, will be used in the future, so this proposal allows for as many types as the implementation sees fit.

§ 4. C Compatibility

The C standards committee, WG14, has added a new annex containing significant extensions to floating-point support to the next revision of the C standard, C23. The annex has not been merged into the C draft standard yet, but text that is very close to what will be in the standard is available in [\[WG14-N2601\]](#). The changes being worked on for C are mostly compatible with the changes proposed for C++ in this proposal. Users will be able to write code that uses IEEE floating-point types, including 16-bit binary, that compiles and behaves the same in both languages.

The C proposal adds optional types `_FloatN`, where N is 16, 32, 64, 128, or greater than 128 and divisible by 32. `_FloatN` is an IEEE binary floating-point type with the given size. These types will have the same representation as the named aliases proposed [below](#). (Except that C does not define a type for the non-IEEE `bf16` format.)

There are two areas of divergence between the C and C++ proposals that are worth discussing:

1. Names: The C proposal uses `_Float16`, `_Float32`, `_Float64`, and `_Float128` as optional keywords naming the IEEE types. This paper proposes type aliases in the `std` namespace, `std::float16_t`, `std::float32_t`, `std::float64_t`, and `std::float128_t`. Since C++ likes to have all its library names in namespace `std`, and C does not have namespace `std` at all, it seems unavoidable that there will be some divergence in this area. See [§ 7.3.1 C compatibility](#) for discussion of the impact of this difference and some possible ways to deal with it.
2. Implicit conversions: In this C++ proposal, narrowing conversions between floating-point types have to be explicit. (See [§ 5.5 Implicit conversions](#)) In the C proposal, conversions between floating-point types can be done implicitly, even when they are narrowing and potentially lossy. This will result in floating-point code that will compile as C but not as C++. While this divergence is unfortunate, it is acceptable because conversions involving extended floating-point types that compile successfully in both languages will behave the same in both languages.

Previously, there was also a difference in usual arithmetic conversions. This proposal and C have always agreed on the results of a binary operator when at least one of the operands is a floating-point type and the two types have different representations. However, when the two operands were different floating-point types with the same representation, this paper proposed that `double + std::float64_t` (assuming they have the same representation) would have type `double`, while in C, `double + _Float64` has type `_Float64`. The rationale for the C rules is that if a user buys into the fixed-layout types explicitly, we should preserve that decision through expressions and library function calls.

This matter was discussed during an SG22 meeting, and a consensus was reached that this paper should instead adopt the C rules; now, with this revision, the result of `double + std::float64_t` is `std::float64_t`.

(C23 will define the term *extended floating types* ([\[WG14-N2601\]](#) section X.2.3) to mean something completely different from the term *extended floating-point types* as used in this paper ([§ 5.2 Extended floating-point types](#)). The terms are only used in specifications and do not appear in user code, so any confusion will hopefully be limited to committee members and not be a problem in the broader programming community. It might be worth the effort to come up with a different name to use in the C++ standard, since "extended" fits the C usage better than the C++ usage.)

§ 5. Core language changes

§ 5.1. Things that aren't changing

It is currently implementation-defined whether or not the floating-point types support infinity and NaN. That is not changing. That feature will still be implementation-defined, even for extended floating-point types.

The radix of the exponent of each floating-point type is currently implementation-defined. That is not changing. This paper will make it easier for the radix of extended floating-point types to be different from the radix of the standard types, allowing implementations to support decimal floating-point while the standard floating-point types remain binary floating-point types.

§ 5.2. Extended floating-point types

Wording: § 9.2.2 [Extended floating-point types](#)

In addition to the three standard floating-point types, `float`, `double`, and `long double`, implementations may define any number of *extended floating-point types*, similar to how implementations may define extended integer types.

An extended floating-point type may have the same representation and the same set of values as a standard floating-point type. But the extended floating-point type is still a separate type, and is not just an alias for the standard type. See § 7.5 [Aliasing standard types](#) for the reasoning behind this decision. It is expected that this will be a common occurrence in implementations that support extended floating-point types.

§ 5.2.1. Reasoning

The set of floating-point types that have hardware support is not possible to accurately predict years into the future. The standard needs to provide an extensible solution so that implementations can adapt to changing hardware without having to modify the standard.

§ 5.3. Conversion rank

Wording: § 9.2.3 [Conversion rank](#)

Define *floating-point conversion rank* to mimic in some ways the existing integer conversion rank. Floating-point conversion rank is defined in terms of the sets of values that the types can represent. If the set of values of type `T` is a strict superset of the set of values of type `U`, then `T` has a higher conversion rank than `U`. If the sets of values of two types are neither a subset nor a superset of each other, then the conversion ranks of the two types are unordered. Two standard floating-point types always have different conversion ranks. But two extended floating-point types, or an extended floating-point type and a standard floating-point type, with the same set of values have the same conversion rank. Floating-point conversion rank forms a partial order, not a total order; this is the biggest difference from integer conversion rank.

When two types have the same conversion rank, they are still ordered by a conversion subrank. The subrank forms a total order among types with the same rank. The IEEE types listed in [§ 7.2 Supported formats](#) have a subrank greater than any standard type with the same rank. The subrank order is otherwise implementation defined.

When two or more standard types have the same representation, then any extended types with that same representation have the same conversion rank as `double`.

§ 5.3.1. Reasoning

Splitting the ranking of floating-point types into rank and subrank simplifies the wording in other places in the standard. Several places where the standard wording would have had to say something like "greater conversion rank or same set of values" can say instead "greater or equal conversion rank." The phrase "set of values" is needed only when defining conversion rank and subrank, and is not used anywhere else when discussing extended floating-point types.

The rules for subrank order enable C++ and C23 to have the same usual arithmetic conversion rules. In C23, types that represent IEEE interchange formats (named `_FloatN` in C23) are preferred over standard types with the same representation, and standard types are preferred over types that represent IEEE extended formats (named `_FloatNx` in C23) with the same representation. The IEEE types listed in [§ 7.2 Supported formats](#) represent IEEE interchange formats, so their subrank is defined to be greater than the subrank of a standard type. This proposal doesn't try to classify any other extended floating-point types as IEEE interchange formats, or IEEE extended formats, or as anything else. So the rest of subrank ordering is implementation defined, leaving it up to quality-of-implementation to match C's behavior if there are any C++ extended floating-point types that represent IEEE extended formats.

Earlier versions of this proposal used the range of finite values to define conversion rank, and had the conversion rank be a total ordering. Discussions in [SG6](#) in Kona 2019 pointed out that that definition

resulted in undesirable interactions between IEEE `binary16` with 5-bit exponent and 10-bit mantissa, and `bfloat16` with 8-bit exponent and 7-bit mantissa. `bfloat16` has a much larger finite range, so it would have a higher conversion rank under the old rules. Mixing `binary16` and `bfloat16` in an arithmetic operation would result in the `binary16` value being converted to `bfloat16` despite the loss of three bits of precision. This implicit loss of precision was worrisome, so the definition of conversion rank was changed so that the usual arithmetic conversions between two floating-point values always preserves the value exactly.

For the purposes of conversion rank, infinity and NaN are treated just like any other values. If type `T` supports infinity and type `U` does not, then `U` can never have a greater conversion rank than `T`, even if `U` has a bigger range and a longer mantissa.

§ 5.4. Promotion

Floating-point promotions are unchanged. For backward compatibility, a conversion from `float` to `double` is considered to be a promotion rather than a standard conversion during overload resolution. But no other floating-point conversions are promotions. There are no changes to the wording for floating-point promotions.

Earlier versions of this proposal promoted function arguments of extended floating-point types that were smaller than `double` (as defined by conversion rank) to `double` when passed as the ellipsis part of a varargs function. The C committee considered this behavior, and for a while it was also a part of the proposed changes for C23. But WG14 argued against this, saying that promotion from `float` to `double` was a holdover from K&R C and should not be extended to new types. This part of the C23 proposal for floating-point was withdrawn. To minimize divergence between C and C++, this was also withdrawn from the C++ proposal.

§ 5.5. Implicit conversions

Wording: § 9.2.4 [Implicit conversions](#)

A conversion between two floating-point types, when at least one of the types is an extended floating-point type, is implicit only if the destination type has greater or equal conversion rank than the source type. Any implicit conversion will be lossless and preserve the value exactly. Any conversion that is potentially lossy must be explicit.

Not all lossless conversions will be implicit, but the situations where a lossless conversion has to be explicit will be relatively rare. It will only happen when two standard types have the same representation and there is also an extended type with that representation. For example, when `double`

and `long double` are both IEEE 64-bit types, the conversion from `long double` to `std::float64_t` would be from a higher conversion rank to a lower rank and therefore is not an implicit conversion, even though the two types have the set of values. This behavior helps maintain consistency when porting among implementations that have different formats for `long double`.

The conversion rules for standard floating-point types can't be changed without breaking existing code, so conversions from `double` to `float` and from `long double` to `double` or `float` will still be implicit.

§ 5.5.1. Reasoning

The standard currently allows implicit conversions between any arithmetic types (except during brace init, when narrowing conversion rules apply), even if the conversion could result in a loss of information. This rule makes it too easy to write buggy code. Changing rules for existing types is not feasible because it would be a major breaking change. But the rules can be changed when types are used in new ways, as was done for brace init and narrowing conversions, or for new types, as is proposed here.

This was discussed in EWG in Prague, and there was consensus to limit implicit conversions for extended floating-point types. "Extended floating point types match the current C++ rules for conversions." 2-3-6-19-3 "Implicit conversions are only allowed if non-narrowing." 14-15-8-0-1

§ 5.5.2. Constant values

A drawback of this part of the proposal is that constant values don't get any special treatment. As a result, this code:

```
std::float16_t x = 1.0;
```

would be ill-formed. The constant `1.0` has type `double`, which cannot be implicitly converted to type `std::float16_t`, even though the value `1.0` can be represented exactly in both types. To compile, the code must have an explicit cast, or, preferably, use a literal suffix:

```
std::float16_t x = 1.0f16;
```

We don't see a good solution to this issue. There are a couple of ways to make floating-point constants usable in more situations, but they both have drawbacks.

§ 5.5.2.1. Standard conversion of constants

It would be possible to change the rules for standard conversions in [conv.double] so that conversions from constant expressions are standard conversions. This would allow all forms of initialization, and would allow the use of floating-point literals in other contexts as well.

```
void f(std::float16_t);
void g() {
    std::float16_t a = 1.0; // OK
    std::float16_t b(2.0);  // OK
    std::float16_t c{3.0};  // OK
    a = 4.0;                // OK
    f(5.0);                 // OK
}
```

This approach looks good at first, but it has some interesting effects on overload resolution. (Thank you to Davis Herring for noticing this and coming up with these examples.) Some function overloads would be viable when passed a constant but not viable when passed a non-constant expression of the same type. (There is precedent for that, with an integer literal `0` being convertible to any pointer type, but that causes confusion and is not a precedent that should be followed.) This behavior is particularly problematic with forwarding functions:

```
// if constant-value conversions were implicit ...
void f(std::float16_t x) { /* ... */ }
template <class T> void call_f(T x) { f(x); }
void g() {
    f(1.0);          // OK, constant-value conversion is implicit
    call_f(1.0);     // error, call to f no longer has constant value
}
```

This can have puzzling effects in real world code:

```
// if constant-value conversions were implicit ...
std::vector<std::float16_t> v;
v.push_back(1.0);    // OK, no forwarding involved
v.emplace_back(1.0); // error, emplace_back forwards to constructor
```

This behavior would be confusing to users, probably more confusing than always remembering to use a cast or a literal suffix when initializing a smaller floating-point type.

§ 5.5.2.2. Direct initialization with constants

We could avoid the problems with overload resolution and forwarding functions by not changing standard conversions, but instead adding a new item to [dcl.init.general]/p16 that would allow a floating-point constant to be used during direct-initialization of an object of a floating-point type. This would allow direct initialization, with `()` or `{}`, but would leave other constructs ill-formed:

```
void f(std::float16_t);
void g() {
    std::float16_t a = 1.0; // error
    std::float16_t b(2.0);  // OK
    std::float16_t c{3.0};  // OK
    a = 4.0;                // error
    f(5.0);                 // error
}
```

The commonly-used C-style initialization with `=` would still be ill-formed because it is a copy-initialization, not a direct-initialization.

This distinction would be confusing to users, probably more confusing than always remembering to use a cast or a literal suffix when initializing a smaller floating-point type.

So we are not proposing any special treatment of floating-point literals or other constant expressions when it comes to implicit conversions. Users will have to use a cast or a literal suffix when initializing, assigning to, or passing a function argument of a smaller-than-`double` extended floating-point type.

§ 5.6. Usual arithmetic conversions

Wording: § 9.2.5 Usual arithmetic conversions

The proposed usual arithmetic conversions for floating-point types are based on the floating-point conversion rank, similar to integer arithmetic conversions. But because floating-point conversions are a partial ordering, there may be some expressions where neither operand will be converted to the other's type. It is proposed that these situations are ill-formed. For the cases where two different types have the same conversion rank, the floating-point conversion subrank is used to determine the result type.

§ 5.6.1. Example

Note: In all the examples in this paper, `float` and `double` are IEEE 32-bit and 64-bit types, `std::floatN_t` is an extended floating-point type for IEEE N -bit, and `std::bfloat16_t` is `bfloat16`.

```
float f32 = 1.0;
std::float16_t f16 = 2.0;
std::bfloat16_t b16 = 3.0;
f32 + f16; // okay, f16 converted to "float", result type is "float"
f32 + b16; // okay, b16 converted to "float", result type is "float"
f16 + b16; // error, neither type can convert to the other via arithmetic c
```

§ 5.7. Narrowing conversions

Wording: § 9.2.6 [Narrowing conversions](#)

A narrowing conversion is a conversion from a type with a higher floating-point conversion rank to a type with a lower conversion rank, or a conversion between two types with unordered conversion rank.

When extended floating-point types are involved, the rules for what is a non-narrowing conversion are exactly the same as the rules for an implicit conversion. A narrowing conversion cannot be done implicitly even in contexts where the narrowing conversion rules don't apply.

To preserve backward compatibility, the rules for non-narrowing conversions and implicit conversions are different when both types are standard types. Conversions from `double` to `float` and from `long double` to `double` are still narrowing conversions.

§ 5.7.1. Interaction with implicit conversions

Because the rules for implicit conversions and narrowing conversions are the same when an extended floating-point type is involved, the rules for narrowing conversions are never actually applied. The rules for narrowing conversions are only checked when there is a valid implicit conversion sequence. Every implicit conversion between floating-point types where at least one of the types is an extended floating-point type is a non-narrowing conversion.

Because of this odd interaction with implicit conversions, the existing wording for narrowing conversions is correct. But we are still proposing that the wording be changed. The new wording does a better job of describing what a narrowing conversion is, and it is more robust if implicit conversions or standard conversions for floating-point types were to change someday.

§ 5.8. Overload resolution

Wording: § 9.2.7 [Overload resolution](#)

When comparing conversion sequences that involve floating-point conversions, prefer conversions that are value-preserving, and prefer conversions to other floating-point types with the same conversion rank if value-preserving conversions are ambiguous.

This is a departure from the previously proposed overload resolution rules, one that improved Evolution consensus by removing a strong opposition to the previously proposed changes in overload resolution. The currently proposed rule has previously been described in the paper as one of the possible alternative designs, though it was never discussed extensively in earlier meetings.

With the proposed change to implicit conversions, preferring value-preserving conversions over lossy conversions comes for free, since overloads with lossy conversions won't be viable candidates (except when both types are standard floating-point types).

Preferring a conversion to a type with the same conversion rank comes from the desire for a function call to be well-formed rather than ambiguous when an overload with a matching representation, but not a matching type, exists.

§ 5.8.1. Examples

```
void f(std::float32_t);
void f(std::float64_t);

f(std::float16_t(1.0)); // ambiguous
f(float(2.0));          // calls std::float32_t, because same conversion rank
f(double(3.0));         // calls std::float64_t, only viable candidate
```

In the call `f(float(2.0))`, both overloads are viable because `float` to `std::float32_t` and `float` to `std::float64_t` are standard conversions. The call would be ambiguous under the existing rules for determining the best conversion. But with the special rules for floating-point types in

this proposal, the conversion from `float` to `std::float32_t` is preferred over the conversion from `float` to `std::float64_t` because `float` and `std::float32_t` have equal conversion ranks while `float` and `std::float64_t` do not.

In the first call, `f(std::float16_t(1.0))`, both overloads are viable. But because none of the types involved, `std::float16_t`, `std::float32_t`, and `std::float64_t`, have equal conversion ranks, the special rules for floating-point types don't apply, and the calls remains ambiguous.

The overload resolution rules are also used to choose between user-defined conversions, as in this example:

```
struct S {
    operator std::float32_t() const;
    operator std::float64_t() const;
};
void f(S s) {
    double x(s); // calls operator std::float64_t, because same conversion ra
}
```

Both conversion sequences, `S` to `std::float32_t` to `double`, and `S` to `std::float64_t` to `double`, are viable. The second sequence is chosen because the standard conversion from `std::float64_t` to `double` is better than the one from `std::float32_t` to `double`, because `std::float64_t` and `double` have equal conversion ranks.

See [§ 5.8.3 Comparisons](#) for more examples.

§ 5.8.2. Alternate proposals

Below, we present two alternate designs: the first one describes what used to be the proposal in the paper, and one that discusses the pitfalls of making no changes to overload resolution rules.

This issue was debated in EWG in Prague, and the first alternative below was favored, but not by enough to consider it consensus given the significant number of neutral and strongly-against votes. "Prefer smaller safe conversions over larger safe conversions in overload resolution." 3-14-10-0-7

The issue was discussed again on a Language Evolution [telecon](#) in June 2020. There were two polls, one a repeat of Prague's poll, with conflicting results. "Prefer smaller safe conversions over larger safe conversions in overload resolution (proposal in the paper, polled in prague)." 0-8-3-4-1 "Overload

resolution should stay the same, two different safe conversions should remain ambiguous (keep the current status-quo)." 5-4-3-4-1

§ 5.8.2.1. *Prefer smallest safe conversions*

When comparing conversion sequences that involve floating-point conversions, prefer conversions that are value-preserving, and prefer conversions to lower conversion ranks over conversions to higher conversion ranks.

With the proposed change to implicit conversions, preferring value-preserving conversions over lossy conversions comes for free, since overloads with lossy conversions won't be viable candidates (except when both types are standard floating-point types).

Preferring a conversion to a smaller type over a conversion to a larger type comes from the desire for a function call to be well-formed rather than ambiguous when there are multiple value-preserving conversions available.

```
void f(std::float32_t);
void f(std::float64_t);

f(std::float16_t(1.0)); // calls std::float32_t, due to smaller conversion
f(float(2.0));          // calls std::float32_t, due to smaller conversion
f(double(3.0));         // calls std::float64_t, only viable candidate
```

The behavior of preferring smaller-distance conversions over longer-distance conversions is not a new idea. It was proposed for integer types in 2012 in [\[N3387\]](#). It was proposed for user-defined types in [\[P1818\]](#).

§ 5.8.2.2. *No change*

The other alternative is to not change the overload resolution rules at all. There would be no disambiguation between standard conversions, so any call with multiple viable function overloads with no exact match would be ambiguous.

```
void f(std::float32_t);
void f(std::float64_t);
```

```
f(std::float16_t(1.0)); // ambiguous
f(float(2.0));          // ambiguous
f(double(3.0));         // calls std::float64_t, only viable candidate
```

§ 5.8.3. Comparisons

The following table shows how various function calls would be resolved under the overload resolution schemes discussed in this section. "Ambiguous" means the call is ill-formed because there are multiple viable functions but none is preferred over the others. "No match" means the call is ill-formed because none of the functions are viable.

Assume that `float` and `double` are 32-bit and 64-bit IEEE floating-point respectively, which is true on most major implementations. Assume that `long double` is X87 80-bit, which is true for most Linux x86 compilers. The types in `std::` are the type aliases described in [§ 7 Type aliases](#).

Assume the following variable declarations:

```
std::bfloat16_t bf_v;
std::float16_t  f16_v;
std::float32_t  f32_v;
std::float64_t  f64_v;
std::float128_t f128_v;
float           float_v;
double          double_v;
long double     ld_v;
```

Assume the following function declarations:

```
void a(float);
void a(double);
void a(long double);

void b(std::float32_t);
void b(std::float64_t);
void b(std::float128_t);
```

Function call	Prefer smallest safe conversion (formerly proposed)	Prefer same conversion rank (currently proposed)	No preference (existing behavior)
---------------	---	--	--------------------------------------

<code>a(bf_v);</code>	<code>a(float)</code>	ambiguous	ambiguous
<code>a(f16_v);</code>	<code>a(float)</code>	ambiguous	ambiguous
<code>a(f32_v);</code>	<code>a(float)</code>	<code>a(float)</code>	ambiguous
<code>a(f64_v);</code>	<code>a(double)</code>	<code>a(double)</code>	ambiguous
<code>a(f128_v);</code>	no match	no match	no match
<code>a(float_v);</code>	<code>a(float)</code>	<code>a(float)</code>	<code>a(float)</code>
<code>a(double_v);</code>	<code>a(double)</code>	<code>a(double)</code>	<code>a(double)</code>
<code>a(ld_v);</code>	<code>a(long double)</code>	<code>a(long double)</code>	<code>a(long double)</code>
<code>b(bf_v);</code>	<code>b(std::float32_t)</code>	ambiguous	ambiguous
<code>b(f16_v);</code>	<code>b(std::float32_t)</code>	ambiguous	ambiguous
<code>b(f32_v);</code>	<code>b(std::float32_t)</code>	<code>b(std::float32_t)</code>	<code>b(std::float32_t)</code>
<code>b(f64_v);</code>	<code>b(std::float64_t)</code>	<code>b(std::float64_t)</code>	<code>b(std::float64_t)</code>
<code>b(f128_v);</code>	<code>b(std::float128_t)</code>	<code>b(std::float128_t)</code>	<code>b(std::float128_t)</code>
<code>b(float_v);</code>	<code>b(std::float32_t)</code>	<code>b(std::float32_t)</code>	ambiguous
<code>b(double_v);</code>	<code>b(std::float64_t)</code>	<code>b(std::float64_t)</code>	ambiguous
<code>b(ld_v);</code>	<code>b(std::float128_t)</code>	<code>b(std::float128_t)</code>	<code>b(std::float128_t)</code>

§ 5.9. Pointer conversions

The proposal of allowing implicit conversions between pointers to two different floating-point types that have the same representation was voted down by EWG in Prague, so it has been withdrawn from this proposal. Allowing the implicit pointer conversions would have eased the transition from using the standard floating-point types to the new named floating-point types. But it complicated the language in a non-obvious way, and the group decided that the benefit was not worth the cost.

§ 5.10. Literal suffixes

To improve usability and compatibility with C, define floating-point literal suffixes for five extended floating-point types. Since extended floating-point types are optional, each of these suffixes are conditionally-supported. The suffixes, both lower-case and upper-case versions, and their corresponding types are:

Suffix	Type
<code>f16</code> or <code>F16</code>	<code>std::float16_t</code>
<code>f32</code> or <code>F32</code>	<code>std::float32_t</code>
<code>f64</code> or <code>F64</code>	<code>std::float64_t</code>
<code>f128</code> or <code>F128</code>	<code>std::float128_t</code>

<code>bf16</code> or <code>BF16</code> <code>std::bfloat16_t</code>

The suffixes do not cause ambiguities with hexadecimal floating-point literals because the exponent in hex literals, a `p` followed by decimal digits, is not optional. That separates the suffix from the main part of the literal and prevents the `f` in the suffix from being mistaken for hex digit.

See [§ 7.4 Literal suffixes](#) for more discussion and the reasons behind this design. See [§ 9.2.1 Literal suffixes](#) for the wording.

§ 5.11. Feature test macro

We are not proposing a predefined language-level feature test macro, like those listed in [cpp.predefined], because such a macro would not be useful. Because extended floating-point types are entirely optional, users can't do anything useful with such a macro. Portable code can conditionally use the standard names for certain extended floating-point types (see [§ 7 Type aliases](#)), but those types will each have their own feature test macro that code can check. Any code that uses extended floating-point types other than those with standard names will be tied to a particular implementation and won't be portable. A standard feature test macro won't help those users know which extended floating-point types are available or what their names are.

The literal suffixes for extended floating-point types are only useful if the names for the types are supported by that implementation. So the availability of each of the literal suffixes is covered by the feature test macro for the type name. (See [§ 7.7 Feature test macros](#)) Separate feature test macros for each of the literal suffixes would not be useful.

§ 6. Library changes

Making extended floating-point types easy to use does not require introducing any new names to the standard library. But it does require adding new overloads or new template specializations in several places. Some of the extended floating-point types will have standard names. Those new names are covered in [§ 7 Type aliases](#).

I/O of extended floating-point types can be done via I/O streams (with some limitations), `std::format`, or `to_chars/from_chars`. Changes are proposed to `<ostream>`, `<istream>`, and `<charconv>` to support this. No changes are necessary to `<format>` because it already refers to all *arithmetic types*.

Implementations will have to change `std::numeric_limits` and `std::is_floating_point` to give correct answers for extended floating-point types. The existing wording in the standard already

covers that (by referring to all *floating-point types* without listing them explicitly), so no wording changes are needed. `std::strong_order` and `std::weak_order` in `<compare>` are similarly covered by generic floating-point wording, so no wording change is needed there either.

Most of the standard functions that operate on floating-point types need wording changes to add overloads or template specializations for the extended floating-point types. These classes and functions are in `<cmath>`, `<complex>`, and `<atomic>`.

No changes are proposed to the following parts of the standard library:

- `<cfloat>`: The header `<cfloat>` provides macros describing some of the properties of the standard floating-point types. The use of macros does not extend very well to extended floating-point types with implementation-specific names. Users should use `std::numeric_limits` rather than macros from `<cfloat>` to query the properties of extended floating-point types.
- The `printf` and `scanf` families of functions: There is no practical way to add specifiers for implementation-specific types with implementation-specific names. C23 will not provide `printf` and `scanf` support for its non-standard floating-point types, so there is no C standard library example to borrow from or build on for this proposal.
- The `strtod` and `stod` families of functions: With different names for each floating-point type (which for `strtod` was inherited from C), that scheme doesn't work well for extended floating-point types.
- The `std::to_string` family of functions: They are defined in terms of `snprintf`, which will not support extended floating-point types.
- `<random>`: [rand.req] states that certain template arguments have to be `float`, `double`, or `long double`. The wording could be changed to allow any floating-point type, but `<random>` does not support extended integral types, so we are not proposing that it be required to support extended floating-point types either.

WG14 is adding optional support for additional floating-point types in an annex to C23. (See [§ 4 C Compatibility](#).) C++ users will eventually see support for some of C++'s extended floating-point types through macros defined in `<cfloat>` and conversion functions in `<cstdlib>`. This proposal is not suggesting identical changes ahead of C23 in these areas. The changes will come to C++ when C++ is rebased on top of C23's standard library.

§ 6.1. Possible new names

While no new names need to be added to the standard library for extended floating-point types to be useful, some new things that could be useful were considered. The authors decided that they are not

useful enough to be worth adding to the standard library. They can easily be added later if it turns out that we were wrong about their usefulness.

§ 6.1.1. Standard/extended floating-point traits

`std::is_floating_point_v<T>` is true for both standard and extended floating-point types. It might be nice if there were traits for `std::is_standard_floating_point` and/or `std::is_extended_floating_point`. But it is not clear why user code would want to distinguish between standard types and extended types. If code needs to do that, a user-defined trait for detecting standard floating-point types can be written easily enough with something like `std::is_same_v<T, float> || std::is_same_v<T, double> || std::is_same_v<T, long double>`.

§ 6.1.2. Conversion rank trait

A type trait that compares the conversion rank of two floating-point types would be useful in situations where generic code needs to know if conversions between the types are safe. See the [constructors](#) for `std::complex` as an example of this.

But we are not proposing that such a trait be added. The API for this trait is not obvious, because there are five possible results when comparing conversion ranks: unordered, less than, greater than, equal with a lesser subrank, and equal with a greater subrank. We think that many potential uses of the trait could use `std::is_convertible` instead, or even better yet `std::is_convertible_without_narrowing` as proposed in [\[P0870\]](#).

§ 6.2. `<charconv>`

Add overloads for all extended floating-point types for the functions `to_chars` and `from_chars`.

Given how much effort it took to implement `to_chars` and `from_chars` for the existing floating-point types, there is some concern that this requirement will be an excessive burden on implementors. After some research and discussions with STL, we feel that the implementation burden will be manageable.

There are several existing algorithms that can be used to implement `to_chars`, such as Ryu and Dragonbox. The [\[Ryu\]](#) GitHub repository has a reference implementation of the algorithm which covers all the floating point types discussed in [§ 7 Type aliases](#). See `ryu_generic_128.h` for reference.

The [\[Eisel-Lemire\]](#) algorithm can be used to implement `from_chars`. There is no reference implementation for 128-bit floating-point numbers yet, but the underlying algorithm has no fundamental limitation that would prevent its usage for large floating-point types.

Wording: [§ 9.3.2 <charconv>](#)

§ 6.3. `<format>`

No wording changes are necessary for `std::format` to support extended floating-point types. [\[format.formatter.spec\]/p2.3](#) already requires that there be a specialization of `struct formatter` for each arithmetic type, which covers the extended floating-point types.

[\[tab:format.type.float\]](#) in [\[format.string.std\]/p22](#) specifies the behavior of floating-point types in terms of `to_chars`, which will support extended floating-point types.

This proposal does not propose any wording changes to `basic_format_arg` in [\[format.arg\]](#). Specifically, extended floating-point types are not added to the `variant` type of the exposition-only data member `basic_format_arg::value`. Doing so would be difficult to specify. Extended floating-point types can be stored in a `basic_format_arg` via `basic_format_arg::handle`, the same mechanism that is used to deal with user-defined class types. Implementations are free to provide special handling for extended floating-point types if they wish, since that does not affect the user-visible behavior.

§ 6.4. I/O Streams

Add support to `std::ostream` and `std::istream`, via overloaded `operator<<` and `operator>>`, for extended floating-point types whose conversion ranks are less than or equal to `long double`. Types whose conversion ranks are greater than or unordered with `long double` will not be handled by I/O streams.

The streaming operators use the virtual functions `num_put<>::do_put` and `num_get<>::do_get` for output and input of arithmetic types. To fully and properly support extended floating-point types, new virtual functions would need to be added. That would be an ABI break. While an ABI break is not out of the question, it would have strong opposition. This proposal is not worth the effort that would be necessary to get an ABI break through the committee.

Therefore, extended floating-point types are supported as well as possible without changing `num_put` or `num_get`. For any extended floating-point type that is no bigger than `long double`, the extended floating-point value is converted to `float`, `double`, or `long double`, as appropriate, and one of the

existing `do_put` or `do_get` functions is called. For types that are larger than `long double`, there are no existing `do_put` or `do_get` functions that have the necessary range and precision. It is proposed that `operator<<` and `operator>>` for these types be defined as deleted.

Wording: [§ 9.3.3 I/O Streams](#)

§ 6.5. `<cmath>`

Add overloads for extended floating-point types to the functions in `<cmath>`. It is expected that this will be the most used part of the library changes.

Trivial implementations of the math functions for extended floating-point types that are no bigger than `long double` can be done by casting the arguments to a standard floating-point that is at least as big as the extended floating-point type, doing the calculations with the standard floating-point type, then casting the result back down to the extended floating-point type.

The GCC [\[libquadmath\]](#) library contains a reference implementation for `<math.h>` functions with IEEE 128-bit floating-point. However, we do not know of any accuracy analyses for mathematical special functions described in section [\[sf.cmath\]](#) with 128-bit floating-point type arguments.

Wording: [§ 9.3.4 <cmath>](#)

§ 6.6. `<complex>`

Make `std::complex<T>` be well-defined when `T` is an extended floating-point type. The explicit specializations of `std::complex<T>` are removed. The only differences between the explicit specializations was the explicit-ness of the constructors that take a complex number of a different type. This behavior is incorporated into the main class template through `explicit(bool)`.

No literal suffixes are defined for complex numbers of extended floating-point types. Subclause [\[complex.literals\]](#) is unchanged.

Wording: [§ 9.3.5 <complex>](#)

§ 6.7. `<atomic>`

The specification for the integral specializations of `std::atomic` states in [\[atomics.types.int\]](#): "There are specializations of the `atomic` class template for the integral types [*all the standard integral types*], and any other types needed by the typedefs in the header `<cstdint>`."

A similar approach is taken for floating-point types. `std::atomic` has specializations for all the standard floating-point types and for any extended floating-point types that are used for the aliases (§ 7 [Type aliases](#)) defined in the `<stdfloat>` header (§ 7.1 [Header name](#)).

Wording: § 9.3.6 `<atomic>`

§ 6.8. Feature test macro

A library feature test macro that is supposed to indicate that the overloads and template specializations for supported extended floating-point types are present doesn't serve any purpose in a conforming implementation, yet we are proposing one anyway. Portable code that uses the type aliases in `<stdfloat>` will check the feature test macro for each of the types that it uses. If the feature test macro for a type is defined, then it would be reasonable to assume that all the core language support and all the library overloads and template specializations are also available. That leaves no reason to check a library-wide feature test macro, since everything the code might use is already covered by the type-specific feature test macro.

A library-wide feature test macro caters to implementations that want to phase in their support for extended floating-point types. Some implementations might choose to create the `<stdfloat>` header with its type aliases once they have the core language support for extended floating-point types in place, without the full library support. That can be useful to users, especially if there is some library support such as `<cmath>` and `<complex>`.

We propose the feature test macro `__cpp_lib_extended_float`, to be defined once the implementation has a `<stdfloat>` header and has defined all the overloads and template specializations for extended floating-point types required by this proposal. The macro is defined in `<version>` and `<stdfloat>`.

Implementations that have implemented extended floating-point types in the language and have provided a `<stdfloat>` header, but have not finished implementing all the library changes, should not define `__cpp_lib_extended_float`. There is not a standard way for users to find out via standard feature test macros which parts of the library have extended floating-point support and which do not.

Wording: § 9.3.7 [Feature test macros](#)

§ 7. Type aliases

This paper introduces type aliases for several fixed-layout floating-point types. Each alias will be defined only if a type with that layout is supported by the implementation, similar to the `intN_t` and `uintN_t` aliases.

Wording: § 9.3.1 `<stdfloat>`

§ 7.1. Header name

The type aliases proposed here do not fit neatly into any existing header. We are proposing that the type aliases be added to a new header `<stdfloat>`. We are not thrilled with that choice, so we are open to other suggestions. An LEWG mailing list discussion of the header name did not generate much discussion or any clear favorite.

An argument can be made to define the type aliases in `<cfloat>`, since the macros that expose the characteristics of floating-point types, including the C23 `_FloatN` types, are defined in `<float.h>`. There is some precedent for C++ adding new stuff to the C++ versions of C headers, but it is not commonly done and is not the preferred solution.

§ 7.2. Supported formats

We propose aliases for the following layouts:

- [IEEE-754-2008] `binary16` - IEEE 16-bit.
- [IEEE-754-2008] `binary32` - IEEE 32-bit.
- [IEEE-754-2008] `binary64` - IEEE 64-bit.
- [IEEE-754-2008] `binary128` - IEEE 128-bit.
- `bfloat16`, which is `binary32` with 16 bits of precision truncated; see [bfloat16].

`binary32` and `binary64` are the most widely used floating-point types, and are the formats that `float` and `double` have in most implementations. `binary16` is becoming more widely used; see this paper's motivation for details. `binary128` has hardware support in IBM POWER P9 chips. `bfloat16` is used in Google's TPUs and in TensorFlow and has hardware support in NVIDIA's latest GPUs.

The most widely used format that is not in this list is X87 80-bit. Even though there is hardware support for this format in all current x86 chips, it is used most often because it is the largest type

available, not because users specifically want that format.

§ 7.3. Names

Earlier revisions of this proposal listed several different possible naming schemes without arguing for one in particular. After an e-mail discussion of this topic on the LEWG mailing list in September 2021 resulted in a clear favorite among those who expressed an opinion, we are proposing the simplest and most straightforward of the proposed naming schemes, and the one already used by Boost.Math (though Boost does not put them in namespace `std`):

- `std::float16_t`
- `std::float32_t`
- `std::float64_t`
- `std::float128_t`
- `std::bfloat16_t`

People liked the simplicity of "float". Even though "float" can refer to decimal floating-point or non-IEEE floating-point formats, for most programmers IEEE binary floating-point is the first thing that comes to mind with the word "float".

Some of the other formats that were considered but were not adopted are `std::fp::binaryN_t`, `std::fp_binaryN_t`, `std::iec559::binaryN_t`, and `std::iec559_binaryN_t`. While the use of "binary" may be more accurate at distinguishing binary floating-point from decimal floating-point, floating-point arithmetic is not the first thing that comes to most users mind when they read the word "binary".

§ 7.3.1. C compatibility

C23 defines `_Float16`, `_Float32`, `_Float64`, and `_Float128` as optional keywords naming the IEEE types. [WG14-N2601] This paper proposes type aliases in the `std` namespace for those same types. Since C++ likes to have all its library names in namespace `std`, and C does not have namespace `std` at all, it seems unavoidable that there will be some divergence in this area. Code that is intended to be compiled only as C will use the `_FloatN` names, while code that is intended to be compiled only as C++ will likely use the `std::floatN_t` names. It would be nice, however, if code

that is intended to be compiled in both languages could use names that would work in both languages without having to resort to something like:

```
#ifdef __cplusplus
#include <stdfloat>
using my_fp16_t = std::float16_t;
#else
typedef _Float16 my_fp16_t;
#endif
```

C++ implementations could use the `_FloatN` names as the names behind the `std::` aliases, allowing the use of the `_FloatN` names in both languages. I expect that most C++ implementations that support extended floating-point types will do this even if it is not required. We could in theory rely on the quality of implementations to get common names in both languages, but that is not the most satisfying approach.

Another way to get common names is for the C++ standard to require C++ implementations to provide the `_FloatN` names in addition to the `std::floatN_t` names. The `_FloatN` names could be conditionally supported keywords in C++ like they are in C. Or the `_FloatN` names could be type aliases at global scope that are available when any floating-point-related header is included, such as `<math.h>` or `<float.h>`. A discussion about this on the EWG and SG22 mailing lists didn't have any consensus, but there was some support for making the `_FloatN` names available in C++ in some way and some resistance to making them keywords. A poll during an SG22 teleconference had weak consensus for making the C names available in C++, but there wasn't discussion or a poll about how best to do that. This was later polled during an LEWG teleconference, and there was consensus against requiring that the C names be available in C++: "Should we require the C names (`_Float16`, etc...) to be available" 1-1-6-10-1. So this proposal is proceeding without requiring that the C names be available, and without any mention of `_FloatN` names in the C++ standard wording.

C23 will define the typedefs `float_t`, `double_t`, `long_double_t`, and `_FloatN_t`. See X.11 in [WG14-N2601]. `_FloatN_t` is not necessarily a typedef of `_FloatN`; it might be an alias for a different floating-point type depending on the value of `FLT_EVAL_METHOD`. A concern has been raised that the `_FloatN_t` names in C may be confused with the `std::floatN_t` names in C++; users might incorrectly assume that `std::float32_t` is the same type as `_Float32_t`, when instead it is the same type as `_Float32`. The authors acknowledge this concern, but we feel it is not serious enough to justify changing the C++ names. The consistency with the `_t` suffix of many other C++ type aliases is more important than minimizing potential confusion with C type names. This was polled during an SG22 teleconference, and changing `std::floatN_t` to `std::floatN` did not have consensus.

§ 7.4. Literal suffixes

C23 defines literal suffixes for IEEE interchange formats and extended formats, for both binary and decimal floating point. The literal suffixes in C23 that correspond to types defined in this proposal are `f16`, `f32`, `f64`, `f128`, and their upper-case versions `F16`, `F32`, `F64`, and `F128`.

We propose matching literal suffixes for C++: `f16` for `std::float16_t`, `f32` for `std::float32_t`, `f64` for `std::float64_t`, and `f128` for `std::float128_t`. Plus an additional suffix, `bf16` for `std::bfloat16_t`, which is not covered by the C standard.

The original proposal was that the literal suffixes be a library feature, requiring an `#include` and a `using namespace` directive to use them. But during an [LEWG](#) teleconference, there was a strong preference to make the literal suffixes a language feature: "The literal suffixes should be a core language feature." 9-5-4-1-0.

There are multiple advantages to having the literal suffixes be built-in to the language. The most obvious is that they are easier to use, since no `#include` or `using namespace` directive is required. It increases compatibility with C, since no C++-specific setup code is required to enable the literal suffixes. Built-in literal suffixes are more friendly when used in a header file, which will happen, because it doesn't require adding a `using` directive to a header that could possibly interfere with other headers.

Wording: § 9.2.1 Literal suffixes

§ 7.5. Aliasing standard types

This was the most contentious issue with the type aliases in the early stages of this proposal, with strong opinions on both sides. In Cologne, SG6 (Numerics) and LEWGI voted in favor of allowing aliasing of standard types, while EWGI was strongly against the idea. After the Cologne meeting, the authors decided that prohibiting aliases of standard types was the better choice. EWG discussed the issue in Prague and there was very strong consensus for the authors' position. "The new `floatX_t` types aren't aliases for `float` / `double` / `long double`, they are independent types." 23-13-0-2-0

The header `<cstdint>` defines integer type aliases for certain integer types, such as `std::int32_t` and `std::int64_t`. These are similar in many ways to the aliases proposed here. The types in

<cstdint> are allowed to alias standard integer types. That has resulted in compilation errors when users try to create an overload set with both standard types and fixed-layout aliases, such as:

```
int bit_count(int x) { /* ... */ }
int bit_count(std::int32_t x) { /* ... */ }
```

If aliasing of standard types is allowed for the floating-point type aliases, then similar compilation errors will likely result:

```
int get_exponent(double x) { /* ... */ }
int get_exponent(std::float64_t x) { /* ... */ }
```

This is the strongest argument against allowing aliasing of standard types. People who don't find this argument persuasive point out that users should not create overload sets with both standard types and fixed-layout type aliases. An overload set should contain just the standard floating-point types or just the fixed-layout types, but not both. The example above that fails to compile is considered poor design and should not be encouraged.

(The arguments about overload sets apply equally to explicit template specializations.)

Not allowing the aliasing of standard types imposes an implementation burden. If aliasing were allowed, then implementations that don't define any extended floating-point types could define some of the aliases with a little bit of library code that boils down to something like:

```
namespace std {
    using float32_t = float;
    using float64_t = double;
}
```

But when aliasing is not allowed, implementations have to support extended floating-point types in at least the compiler front end, which is not a trivial task. There is also a burden on the name mangling ABI, which will have to define how to encode these extended floating-point types.

The authors feel that the burden on users of allowing aliasing of standard types is greater than the burden on implementers of not allowing such aliasing.

(This issue of aliasing of standard types is tightly bound to the overload resolution rules (§ 5.8 [Overload resolution](#)) for extended floating-point types. If the overload resolution rules are not changed, then having `std::float64_t` be an alias of an extended floating-point type rather than an alias of `double` will cause the following code to not compile:

```
void f(std::float32_t);
void f(std::float64_t);
void g(double x) {
    f(x); // error - ambiguous call without overload resolution changes
}
```

If that code doesn't compile, that would be a bigger burden on users than not being able to overload on both `double` and `std::float64_t`.)

§ 7.6. Layout vs. behavior

The IEEE-conforming type aliases have the specified IEEE layout and the required behavior. For the four IEEE-conforming type aliases, `std::numeric_limits<T>::is_iec559` is true.

§ 7.7. Feature test macros

Since implementations may choose to support (or not) each of the fixed-layout aliases individually, there is a separate test macro for each of the type aliases. The names of the test macros are derived from the names of type alias names. These macros are different from all other library feature test macros in that they are conditionally supported. They don't indicate that the implementation has implemented this proposal; instead they indicate that the type in question is available in this implementation.

The names of the proposed macros are:

- `__cpp_lib_float16_t`
- `__cpp_lib_float32_t`
- `__cpp_lib_float64_t`

- `__cpp_lib_float128_t`
- `__cpp_lib_bfloat16_t`

Wording: § 9.3.7 *Feature test macros*

§ 8. Implementation experience

None, yet. Some implementations have additional floating-point types that are built in to the compiler, such as GCC and Clang's `__float128` on Linux. But those types don't follow the rules for extended floating-point types described in this proposal. They are useful for demonstrating the need for this proposal, but their usefulness as implementation experience is limited.

§ 9. Wording

Wording changes are relative to N4901, dated 2021-10-23.

§ 9.1. References

Because some of the extended floating-point types are required to conform to certain IEEE types, move the IEEE floating-point document reference from the bibliography to the normative references section, and update it from the 2011 version to the 2020 version.

Remove the ISO 60559 entry from the Bibliography:

~~- ISO/IEC/IEEE 60559:2011, *Information technology — Microprocessor Systems — Floating-Point arithmetic*~~

Add the ISO 60559 entry to section 2 "Normative references" [**intro.refs**]:

- ISO/IEC/IEEE 60559:2020, *Information technology — Microprocessor Systems — Floating-Point arithmetic*

Change the text of footnote 196, which is attached to the end of paragraph 57 of section 17.3.5.2 "`numeric_limits` members" [**numeric.limits.members**], which is in the description of member `is_iec559`:

196) ISO/IEC/IEEE 60559: ~~2011~~ 2020 is the same as IEEE 754- ~~2008~~ 2019 .

§ 9.2. Core

§ 9.2.1. Literal suffixes

Design: § 7.4 *Literal suffixes*

In 5.13.4 "Floating-point literals" [**lex.fcon**], change the grammar production for *floating-point-suffix*:

floating-point-suffix: one of

f l f16 f32 f64 f128 bf16 F L F16 F32 F64 F128 BF16

In the same section, change paragraph 1:

The type of a *floating-point-literal* ([**basic.fundamental**] and [**stdfloat.types**]) is determined by its *floating-point-suffix* as specified in Table [tab:lex.fcon.type]. The floating-point suffixes f16, f32, f64, f128, bf16, F16, F32, F64, F128, and BF16 are conditionally-supported.

Add five new rows to the end of Table 11: Types of *floating-point-literals* [tab:lex.fcon.type].

<i>floating-point-suffix</i>	type
none	double
f or F	float
l or L	long double
f16 or F16	std::float16_t
f32 or F32	std::float32_t
f64 or F64	std::float64_t
f128 or F128	std::float128_t
bf16 or BF16	std::bfloat16_t

§ 9.2.2. Extended floating-point types

Design: § 5.2 *Extended floating-point types*

Modify 6.8.2 "Fundamental types" [**basic.fundamental**] paragraph 12:

The three distinct types `float`, `double`, and `long double` can represent floating-point numbers. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The types `float`, `double`, and `long double`, and cv-qualified versions ([`basic.type.qualifier`]) thereof, are collectively termed *standard floating-point types*.

There may also be additional implementation-defined types, distinct from the standard floating-point types, that can represent floating-point numbers. These additional types, if any, and the cv-qualified versions thereof are collectively termed *extended floating-point types*. The standard and extended floating-point types are collectively termed *floating-point types*. The value representation of floating-point types is implementation-defined.

[*Note*: This document imposes no requirements on the accuracy of floating-point operations; see also [`support.limits`]. — *end note*]

Integral and floating-point types are collectively termed *arithmetic types*. Specializations of the standard library template `std::numeric_limits` [`support.limits`] shall specify the maximum and minimum values of each arithmetic type for an implementation.

Editorial note: Is the note still accurate? Should it be changed to refer only to standard floating-point types, since the `std::floatN_t` types must be IEEE-conforming?

§ 9.2.3. Conversion rank

Design: § 5.3 Conversion rank

Change the title of section 6.8.5 [`conv.rank`] from "`Integer conversion rank`" to "`Conversion ranks`", but leave the stable name unchanged. Insert new paragraphs at the end of the subclause:

Every floating-point type has a *floating-point conversion rank* defined as follows:

- The rank of a floating-point type T is greater than the rank of any floating-point type whose set of values is a proper subset of the set of values of T.
- The rank of `long_double` is greater than the rank of `double`, which is greater than the rank of `float`.
- Two extended floating-point types with the same set of values have equal ranks.
- An extended floating-point type with the same set of values as exactly one standard floating-point type has a rank equal to the rank of the standard floating-point type.
- An extended floating-point type with the same set of values as more than one standard floating-point type has a rank equal to the rank of `double`.

[Note: The conversion ranks of floating-point types T1 and T2 are unordered if the set of values of T1 is neither a subset nor a superset of the set of values of T2. This happens when one type has both a larger range and a lower precision than the other. -- end note]

Floating-point types that have equal floating-point conversion ranks are ordered by *floating-point conversion subrank*. The subrank forms a total order among types with equal ranks. The types `std::float16_t`, `std::float32_t`, `std::float64_t`, and `std::float128_t` (`[stdfloat.types]`) have a greater conversion subrank than any standard floating-point type with equal conversion rank. Otherwise, the conversion subrank order is implementation defined.

[Note: The floating-point conversion rank and subrank are used in the definition of the usual arithmetic conversions (`[expr.arith.conv]`). -- end note]

§ 9.2.4. Implicit conversions

Design: § 5.5 Implicit conversions

Modify section 7.3.10 "Floating-point conversions" [**conv.double**] as follows:

A prvalue of floating-point type can be converted to a prvalue of another floating-point type with a greater or equal conversion rank ([conv.rank]). A prvalue of standard floating-point type can be converted to a prvalue of another standard floating-point type .

If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

The conversions allowed as floating-point promotions are excluded from the set of floating-point conversions.

In section 7.6.1.9 "Static cast" [**expr.static.cast**], add a new paragraph after paragraph 10 ("A value of integral or enumeration type can [...]):

A prvalue of floating-point type can be explicitly converted to any other floating-point type. If the source value can be exactly represented in the destination type, the result of the conversion has that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

Editorial note: A **static_cast** from a higher floating-point conversion rank to a lower conversion rank is already covered by [expr.static.cast] p7, which talks about inverses of standard conversions. The new paragraph is necessary to allow explicit conversions between types with unordered conversion ranks. The wording about what to do with the value is stolen from the floating-point conversions section [conv.double].

§ 9.2.5. Usual arithmetic conversions

Design: [§ 5.6 Usual arithmetic conversions](#)

Modify section 7.4 "Usual arithmetic conversions" [**expr.arith.conv**] as follows:

Editorial note: This includes a drive-by fix of removing "shall" from otherwise unchanged parts of this section.

- 1 Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
 - (1.1) - If either operand is of scoped enumeration type ([dcl.enum]), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
 - (1.2) - ~~If either operand is of type long double, the other shall be converted to long double.~~
 - (1.3) - ~~Otherwise, if either operand is double, the other shall be converted to double.~~
 - (1.4) - ~~Otherwise, if either operand is float, the other shall be converted to float.~~
 - (1.5) - Otherwise, if either operand is of floating-point type, the following rules are applied:
 - (1.1) - If both operands have the same type, no further conversion is needed.
 - (1.2) - Otherwise, if one of the operands is of a non-floating-point type, that operand is converted to the type of the operand with the floating-point type.
 - (1.3) - Otherwise, if the floating-point conversion ranks ([conv.rank]) of the types of the operands are ordered but not equal, then the operand of the type with the lesser floating-point conversion rank is converted to the type of the other operand.
 - (1.4) - Otherwise, if the floating-point conversion ranks of the types of the operands are equal, then the operand with the lesser floating-point conversion subrank ([conv.rank]) is converted to the type of the other operand.
 - (1.5) - Otherwise, the expression is ill-formed.
 - (1.6) - Otherwise, the integral promotions ([conv.prom]) ~~shall be~~ are performed on both operands.(59) Then the following rules ~~shall be~~ are applied to the promoted operands:
 - (1.1) - If both operands have the same type, no further conversion is needed.

- (1.2) - Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank ~~shall be~~ **is** converted to the type of the operand with greater rank.
 - (1.3) - Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type ~~shall be~~ **is** converted to the type of the operand with unsigned integer type.
 - (1.4) - Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type ~~shall be~~ **is** converted to the type of the operand with signed integer type.
 - (1.5) - Otherwise, both operands ~~shall be~~ **are** converted to the unsigned integer type corresponding to the type of the operand with signed integer type.
- 2 If one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, this behavior is deprecated (D.2).

§ 9.2.6. Narrowing conversions

Design: § 5.7 Narrowing conversions

Modify the definition of narrowing conversions in 9.4.5 "List-initialization" [**dcl.init.list**] paragraph 7 item 2:

- ~~from long double to double or float, or from double to float~~ **from a floating-point type T to another floating-point type whose floating-point conversion rank is neither greater than nor equal to that of T**, except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

Modify 7.6.8 "Three-way comparison operator" [**expr.spaceship**] paragraph 4.1 as follows, just changing "floating-point type" to "standard floating-point type":

- If a narrowing conversion is required, other than from an integral type to a standard floating-point type, the program is ill-formed.

§ 9.2.7. Overload resolution

Design: [§ 5.8 Overload resolution](#)

In 12.2.4.3 "Ranking implicit conversion sequences" [**over.ics.rank**] paragraph 4, add a new item between (4.2) and (4.3):

- (4.2) A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.
- (4.3) A conversion in either direction between floating-point type FP1 and floating-point type FP2 is better than a conversion between FP1 and arithmetic type T3 if

- (4.3.1) the floating-point conversion rank ([conv.rank]) of FP1 is equal to the rank of FP2, and
- (4.3.2) T3 is not a floating-point type, or T3 is a floating-point type whose rank is not equal to the rank of FP1, or the floating-point conversion subrank ([conv.rank]) of FP2 is greater than the subrank of T3. [Example:

```
int f(std::float32_t);  
int f(std::float64_t);  
int f(long long);  
float x;  
std::float16_t y;  
int i = f(x); // calls f(std::float32_t) on implementations where  
                // float and std::float32_t have equal conversion r  
int j = f(y); // error, ambiguous, no equal conversion rank
```

— end example]

- ~~(4.3)~~ (4.4) If class B is derived directly or indirectly from class A, conversion of B* to A* is better than conversion of B* to void*, and conversion of A* to void* is better than conversion of B* to void*.

Editorial note: The last part of item 4.3.2 that compares the conversion subranks is necessary to handle situations where three or more types have the same conversion rank. This can happen if implementations support extended floating-point types beyond those listed in [§ 7.2 Supported formats](#). For example, the types `double`, `std::float64_t` (a.k.a. `_Float64` in C23), and `_Float32x` (from C23) would have the same conversion rank, and conversions from `_Float32x` to `std::float64_t` would be preferred over conversions from `_Float32x` to `double`, because `std::float64_t` has a greater subrank than `double`.

§ 9.3. Library

§ 9.3.1. `<stdfloat>`

Design: [§ 7 Type aliases](#)

In `[tab:headers.cpp]` "C++ library headers" in section 16.4.2.3 `[headers]`, add a new entry to the table for `<stdfloat>`.

Add a new section to 17 "Language support library" `[support]` at the level of `[cstdint]` just after `[cstdint]`. The section has the stable name `[stdfloat]` with two subsections, `[stdfloat.syn]` and `[stdfloat.types]`.

17.x Floating-point types [stdfloat]

17.x.1 Header <stdfloat> synopsis [stdfloat.syn]

```
namespace std {  
    using float16_t = extended floating-point type; // optional  
    using float32_t = extended floating-point type; // optional  
    using float64_t = extended floating-point type; // optional  
    using float128_t = extended floating-point type; // optional  
    using bfloat16_t = extended floating-point type; // optional  
}
```

The header defines conditionally-supported names for certain extended floating-point types. Each type alias is defined only if the implementation supports the specified extended floating-point type. [Note: On some conforming implementations that do not support any extended floating-point types the <stdfloat> header could contain only an empty namespace definition. -- end note]

17.x.2 Extended floating-point type aliases [stdfloat.types]

ISO/IEC/IEEE 60559 specifies interchangeable formats for binary floating-point types. These formats can be identified by their storage width in bits N , precision in bits p and maximum exponent $emax$.

The following table provides the parameters for extended format types:

Parameter	float16_t	float32_t	float64_t	float128_t	bfloat16_t
N , storage width in bits	16	32	64	128	16
p , precision in bits	11	24	53	113	8
$emax$, maximum exponent	15	127	1023	16383	127
w , exponent field width in bits	5	8	11	15	8

[Note: The precision p includes the implicit 1 bit at the beginning of the mantissa, so the storage used for the mantissa is $p-1$ bits.]

[Note:

- float16_t type adheres to binary16 ISO/IEC/IEEE 60559.

- float32_t type adheres to binary32 ISO/IEC/IEEE 60559.
- float64_t type adheres to binary64 ISO/IEC/IEEE 60559.
- float128_t type adheres to binary128 ISO/IEC/IEEE 60559. -- *end note*]

numeric_limits<floating-point type>::is_iec559 ([numeric.limits.members]) is true when floating-point-type is float16_t, float32_t, float64_t, or float128_t.

§ 9.3.2. <charconv>

Design: § 6.2 <charconv>

Add a new paragraph to the beginning of 20.19.1 "Header <charconv> synopsis" [**charconv.syn**], before the start of the synopsis:

When a function has a parameter of type *integral*, the implementation provides overloads for all cv-unqualified signed and unsigned integer types and char as the parameter type. When a function has a parameter of type *floating-point*, the implementation provides overloads for all cv-unqualified floating-point types as the parameter type.

Change the header synopsis in [**charconv.syn**] as follows:

```
to_chars_result to_chars(char* first, char* last, see-belowintegral va
to_chars_result to_chars(char* first, char* last, floatfloating-point
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);
to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt);
to_chars_result to_chars(char* first, char* last, double value, chars_
to_chars_result to_chars(char* first, char* last, long double value, c
to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                           chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                           chars_format fmt, int precision);

// ...

from_chars_result from_chars(const char* first, const char* last,
                             see-belowintegral& value, int base = 10);

from_chars_result from_chars(const char* first, const char* last, floa
                             chars_format fmt = chars_format::general)
from_chars_result from_chars(const char* first, const char* last, doub
                           chars_format fmt = chars_format::general)
from_chars_result from_chars(const char* first, const char* last, long
                           chars_format fmt = chars_format::general)
```

In 20.19.2 "Primitive numeric output conversion" [**charconv.to.chars**], leave the first three paragraphs

unchanged, but modify the rest of the section as follows:

```
to_chars_result to_chars(char* first, char* last, see belowintegral valu
```

Preconditions: base has a value between 2 and 36 (inclusive).

Effects: The value of value is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 10..35 (inclusive) are represented as lowercase characters a..z. If value is less than zero, the representation starts with ' - '.

Throws: Nothing.

Remarks: [*Note:* The implementation ~~shall provide~~ provides overloads for all cv-unqualified signed and unsigned integer types and char as the type of the parameter value. - end note]

```
to_chars_result to_chars(char* first, char* last, floatfloating-point va  
to_chars_result to_chars(char* first, char* last, double value);  
to_chars_result to_chars(char* first, char* last, long double value);
```

Effects: value is converted to a string in the style of printf in the "C" locale. The conversion specifier is f or e, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of f.

Throws: Nothing.

[*Note:* The implementation provides overloads for all cv-unqualified floating-point types as the type of the parameter value. - end note]

```
to_chars_result to_chars(char* first, char* last, floatfloating-point va  
to_chars_result to_chars(char* first, char* last, double value, chars_fo  
to_chars_result to_chars(char* first, char* last, long double value, cha
```

Preconditions: fmt has the value of one of the enumerators of chars_format.

Effects: value is converted to a string in the style of printf in the "C" locale.

Throws: Nothing.

[*Note: The implementation provides overloads for all cv-unqualified floating-point types as the type of the parameter value. - end note*]

```
to_chars_result to_chars(char* first, char* last, floatfloating-point value,
                        chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                        chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                        chars_format fmt, int precision);
```

Preconditions: fmt has the value of one of the enumerators of chars_format.

Effects: value is converted to a string in the style of printf in the "C" locale with the given precision.

Throws: Nothing.

[*Note: The implementation provides overloads for all cv-unqualified floating-point types as the type of the parameter value. - end note*]

See also: ISO C 7.21.6.1

Modify 20.19.3 "Primitive numeric input conversion" [**charconv.from.chars**] as follows:

- 1 All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`. [*Note*: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note*] Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest`, and the member `ec` is value-initialized.

```
from_chars_result from_chars(const char* first, const char* last,  
                             see belowintegral& value, int base = 10);
```

- 2 *Preconditions*: `base` has a value between 2 and 36 (inclusive).
- 3 *Effects*: The pattern is the expected form of the subject sequence in the "C" locale for the given nonzero base, as described for `strtoul`, except that no "0x" or "0X" prefix shall appear if the value of `base` is 16, and except that ' - ' is the only sign that may appear, and only if `value` has a signed type.
- 4 *Throws*: Nothing.
- 5 *Remarks*: [*Note*: The implementation ~~shall provide~~ provides overloads for all `cv-qualified` signed and unsigned integer types and `char` as the referenced type of the parameter `value`. — *end note*]

```
from_chars_result from_chars(const char* first, const char* last, float&  
                             chars_format fmt = chars_format::general);  
from_chars_result from_chars(const char* first, const char* last, double  
                             chars_format fmt = chars_format::general);  
from_chars_result from_chars(const char* first, const char* last, long d  
                             chars_format fmt = chars_format::general);
```

- 6 *Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

7 *Effects:* The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

(7.1) - the sign '+' may only appear in the exponent part;

(7.2) - if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;

(7.3) - if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and

(7.4) - if `fmt` is `chars_format::hex`, the prefix "0x" or "0X" is assumed. [*Example:* The string 0x123 is parsed to have the value 0 with remaining characters x123. - *end example*]

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

8 *Throws:* Nothing.

9 [*Note:* The implementation provides overloads for all cv-unqualified floating-point types as the referenced type of the parameter `value`. - *end note*]

See also: ISO C 7.22.1.3, 7.22.1.4

§ 9.3.3. I/O Streams

Design: § 6.4 [I/O Streams](#)

§ 9.3.3.1. `<ostream>`

Modify 29.7.5.2.1 "General" [**`ostream.general`**] as follows:

Insert a new paragraph at the beginning of the section, before the synopsis:

When a function has a parameter type `small-ext-fp`, the implementation provides overloads for all cv-unqualified extended floating-point types ([`basic.fundamental`]) whose floating-point conversion rank ([`conv.rank`]) is less than or equal to the conversion rank of `long_double`. When a function has a parameter type `big-ext-fp`, the implementation provides overloads for all cv-unqualified extended floating-point types whose floating-point conversion rank is neither less than nor equal to the conversion rank of `long_double`.

Modify the section of the synopsis for `operator<<` as follows:

```
// [ostream.formatted], formatted output
basic_ostream& operator<<(basic_ostream& (*pf)(basic_ostream&));
basic_ostream& operator<<(basic_ios<charT, traits>& (*pf)(basic_ios<char
basic_ostream& operator<<(ios_base& (*pf)(ios_base&));

basic_ostream& operator<<(bool n);
basic_ostream& operator<<(short n);
basic_ostream& operator<<(unsigned short n);
basic_ostream& operator<<(int n);
basic_ostream& operator<<(unsigned int n);
basic_ostream& operator<<(long n);
basic_ostream& operator<<(unsigned long n);
basic_ostream& operator<<(long long n);
basic_ostream& operator<<(unsigned long long n);
basic_ostream& operator<<(float f);
basic_ostream& operator<<(double f);
basic_ostream& operator<<(long double f);
basic_ostream& operator<<(small-ext-fp f);
basic_ostream& operator<<(big-ext-fp f) = delete;

basic_ostream& operator<<(const void* p);
basic_ostream& operator<<(nullptr_t);
basic_ostream& operator<<(basic_streambuf<char_type, traits>* sb);
```

Modify 29.7.5.3.2 "Arithmetic inserters" [**ostream.inserters.arithmetic**], adding the following at the end of the section:

```
basic_ostream& operator<<(small-ext-fp val);
```

[*Note: *small-ext-fp* is an extended floating-point type whose floating-point conversion rank is less than or equal to the conversion rank of *long_double* ([ostream.general]). -- end note]*

Effects: When *val* is of a type whose floating-point conversion rank is less than or equal to that of *double*, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<  
  num_put<charT, ostreambuf_iterator<charT, traits>>  
    >(getloc()).put(*this, *this, fill(),  
      static_cast<double>(val)).failed();
```

Otherwise the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<  
  num_put<charT, ostreambuf_iterator<charT, traits>>  
    >(getloc()).put(*this, *this, fill(),  
      static_cast<long_double>(val)).failed();
```

If *failed* is true then does *setstate(badbit)*, which may throw an exception, and returns.

Returns: *this.

§ 9.3.3.2. *<istream>*

Modify 29.7.4.2.1 "General" [**istream.general**] as follows:

Insert a new paragraph at the beginning of the section, before the synopsis:

When a function has a parameter type *small-ext-fp*, the implementation provides overloads for all cv-unqualified extended floating-point types ([basic.fundamental]) whose floating-point conversion rank ([conv.rank]) is less than or equal to the conversion rank of *long_double*. When a function has a parameter type *big-ext-fp*, the implementation provides overloads for all cv-unqualified extended floating-point types whose floating-point conversion rank is neither less than nor equal to the conversion rank of *long_double*.

Modify the section of the synopsis for `operator>>` as follows:

```
// [istream.formatted], formatted input
basic_istream& operator>>(basic_istream& (*pf)(basic_istream&));
basic_istream& operator>>(basic_ios<charT, traits>& (*pf)(basic_ios<char
basic_istream& operator>>(ios_base& (*pf)(ios_base&));

basic_istream& operator>>(bool& n);
basic_istream& operator>>(short& n);
basic_istream& operator>>(unsigned short& n);
basic_istream& operator>>(int& n);
basic_istream& operator>>(unsigned int& n);
basic_istream& operator>>(long& n);
basic_istream& operator>>(unsigned long& n);
basic_istream& operator>>(long long& n);
basic_istream& operator>>(unsigned long long& n);
basic_istream& operator>>(float& f);
basic_istream& operator>>(double& f);
basic_istream& operator>>(long double& f);
basic_istream& operator>>(small-ext-fp& f);
basic_istream& operator>>(big-ext-fp& f) = delete;

basic_istream& operator>>(void*& p);
basic_istream& operator>>(basic_streambuf<char_type, traits>* sb);
```

Modify 29.7.4.3.2 "Arithmetic extractors" [**istream.formatted.arithmetic**] add the following at the end of the section:

```
basic_istream& operator>>(small-ext-fp& val);
```

[Note: *small-ext-fp* is an extended floating-point type whose floating-point conversion rank is less than or equal to the conversion rank of *long_double* ([istream.general]). -- end note]

Let *std-fp* be a standard floating-point type:

- if *small-ext-fp* has a floating-point conversion rank that is less than or equal to that of *float*, then *std-fp* is *float*,
- otherwise, if *small-ext-fp* has a floating-point conversion rank that is less than or equal to that of *double*, then *std-fp* is *double*,
- otherwise, *std-fp* is *long_double*.

The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
using numget = num_get<charT, istreambuf_iterator<charT, traits>>;  
iosstate err = ios_base::goodbit;  
std-fp fval;  
use_facet<numget>(loc).get(*this, 0, *this, err, fval);  
if (fval < -numeric_limits<small-ext-fp>::max()) {  
    err |= ios_base::failbit;  
    val = -numeric_limits<small-ext-fp>::max();  
} else if (numeric_limits<small-ext-fp>::max() < fval) {  
    err |= ios_base::failbit;  
    val = numeric_limits<small-ext-fp>::max();  
} else  
    val = static_cast<small-ext-fp>(fval);  
setstate(err);
```

§ 9.3.4. **<cmath>**

Design: § 6.5 **<cmath>**

Modify 26.8.1 "Header **<cmath>** synopsis" [**cmath.syn**] paragraph 2 as follows:

For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there ~~shall be~~ are additional overloads sufficient to ensure:

- ~~1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (6.7.1) corresponding to `double` parameters are effectively cast to `long double`.~~
- ~~2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.~~
- ~~3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.~~
- 1. If any argument corresponding to a `double` parameter has floating-point type, then all arguments of arithmetic type ([basic.fundamental]) corresponding to `double` parameters are effectively cast to the floating-point type with the greatest floating-point conversion rank ([conv.rank]) and greatest floating-point conversion subrank among the types of such floating-point arguments. If two such floating-point arguments have types whose conversion rank is unordered, the program is ill-formed.
- 2. Otherwise, all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.

[Note: `abs` is exempted from these rules in order to stay compatible with C. -- end note]

Modify section 26.8.2 "Absolute values" [`c.math.abs`] as follows:

- 1 [*Note*: The headers `<cstdlib>` and `<cmath>` declare the functions described in this subclause. — *end note*]

```
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);
```

- 2 *Effects*: The `abs` functions that take integer arguments have the semantics specified in the C standard library for the functions `abs`, `labs`, and `llabs`, ~~`fabsf`, `fabs`, and `fabsl`~~.
- 3 *Remarks*: If `abs ( )` is called with an argument of type `X` for which `is_unsigned_v<X>` is `true` and if `X` cannot be converted to `int` by integral promotion, the program is ill-formed. [*Note*: Arguments that can be promoted to `int` are permitted for compatibility with C. — *end note*]

`floating-point abs(floating-point x);`

- 4 *Returns*: The absolute value of `x`.
- 5 *Remarks*: The implementation provides overloads for all cv-unqualified floating-point types as the type of parameter `x`, with the same floating-point type as the return type.

See also: ISO C 7.12.7.2, 7.22.6.1

Modify 26.8.5 "Classification / comparison functions" [**`c.math.fpclass`**] as follows:

The classification / comparison functions behave the same as the C macros with the corresponding names defined in the C standard library. Each function is overloaded for ~~the three~~ all cv-unqualified floating-point types.

See also: ISO C 7.12.3, 7.12.4

§ 9.3.5. `<complex>`

Design: § 6.6 `<complex>`

Modify 26.4.1 "Complex numbers / General" [**complex.numbers.general**] paragraph 2 as follows:

The effect of instantiating the template `complex` for any type ~~other than `float`, `double`, or `long double`~~ that is not a cv-unqualified floating-point type is unspecified. The specializations ~~`complex<float>`, `complex<double>`, and `complex<long double>`~~ of `complex` for cv-unqualified floating-point types are literal types ([basic.types]).

Delete the explicit specializations from 26.4.2 "Header `<complex>` synopsis" [**complex.syn**]:

```
namespace std {  
    // 26.4.2, class template complex  
    template<class T> class complex;  
  
    // 26.4.3, specializations  
    template<> class complex<float>;  
    template<> class complex<double>;  
    template<> class complex<long double>;  
  
    // ...
```

In 26.4.3 "Class template `complex`" [**complex**], modify the synopsis of the constructors as follows:

```
constexpr complex(const T& re = T(), const T& im = T());  
constexpr complex(const complex&) = default;  
template<class X> constexpr explicit(see below) complex(const complex<X>
```

Remove section 26.4.4 "Specializations" [**complex.special**] in its entirety.

In 26.4.5 "Member functions" [**complex.members**], add the following after paragraph 1:

```
template<class X> constexpr explicit(see below) complex(const complex<X>
```

Postconditions: `real(.) == other.real(.)` and `imag(.) == other.imag(.)`.

Remarks: The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T` is greater than or equal to the floating-point conversion rank of `X`.

Modify 26.4.10 "Additional overloads" [**cmplx.over**] paragraphs 2 and 3 as follows:

The additional overloads shall be sufficient to ensure:

- ~~If the argument has type `long double`, then it is effectively cast to `complex<long double>`.~~
- ~~Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.~~
- ~~Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.~~
- If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
- Otherwise, if the argument has integer type, then it is effectively cast to `complex<double>`.

Function template `pow` ~~shall have~~ has additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

- ~~If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.~~
- ~~Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.~~
- ~~Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.~~
- If one argument is of type `T1` or `complex<T1>` and the other argument is of type `T2` or `complex<T2>` where `T1` and `T2` are both floating-point types:
 - If the floating-point conversion ranks (`[conv.rank]`) of `T1` and `T2` are unordered, the program is ill-formed.
 - Otherwise, if `T1` has greater floating-point conversion rank than `T2`, or if `T1` and `T2` have equal conversion ranks and `T1` has greater floating-point conversion subrank (`[conv.rank]`) than `T2`, then both arguments are effectively cast to `complex<T1>`.
 - Otherwise, both arguments are effectively cast to `complex<T2>`.
- Otherwise, if the other argument has integer type, it is effectively cast to `complex<T>`.

§ 9.3.6. `<atomic>`

Design: § 6.7 `<atomic>`

Modify 31.8.4 "Specializations for floating-point types" [`atomics.types.float`] paragraph 1 as follows:

There are specializations of the `atomic` class template for the floating-point types `float`, `double`, and `long double`, and any other floating-point types needed by the type aliases in the header `<stdfloat>`. For each such type *floating-point*, the specialization `atomic<floating-point>` provides additional atomic operations appropriate to floating-point types.

§ 9.3.7. Feature test macros

Design: § 5.11 Feature test macro, § 6.8 Feature test macro, and § 7.7 Feature test macros

Add a new macro to the large table in [**version.syn**]:

```
#define __cpp_lib_extended_float_date // also in <stdfloat>
```

Add the following feature test macros to [**version.syn**]. These are different from all the other library feature test macros because they are conditionally defined, so they don't fit neatly into the existing table. Some guidance is needed on how best to word this.

- `__cpp_lib_float16_t`: if `std::float16_t` is available, also in `<stdfloat>`
- `__cpp_lib_float32_t`: if `std::float32_t` is available, also in `<stdfloat>`
- `__cpp_lib_float64_t`: if `std::float64_t` is available, also in `<stdfloat>`
- `__cpp_lib_float128_t`: if `std::float128_t` is available, also in `<stdfloat>`
- `__cpp_lib_bfloat16_t`: if `std::bfloat16_t` is available, also in `<stdfloat>`

§ References

§ Informative References

[BFLOAT16]

bfloat16 floating-point format. URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

[Eisel-Lemire]

Daniel Lemire. *Number Parsing at a Gigabyte per Second*. URL: <https://arxiv.org/abs/2101.11408>

[IEEE-754-2008]

IEEE Standard for Floating-Point Arithmetic. 29 August 2008. URL:
<http://ieeexplore.ieee.org/servlet/opac?punumber=4610933>

[LIBQUADMATH]

The GCC Quad-Precision Math Library. URL: <https://gcc.gnu.org/onlinedocs/gcc-6.5.0/libquadmath.pdf>

[N1703]

Paul A. Bristow; Christopher Kormanyos; John Maddock. *Floating-Point Typedefs Having Specified Widths*. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1703.pdf>

[N3387]

Jens Maurer. *Overload resolution tiebreakers for integer types*. 12 September 2012. URL: <https://wg21.link/n3387>

[P0192]

Michał Dominiak; et al. *`short float` and fixed-size floating point types*. URL: <https://wg21.link/P0192>

[P0870]

Giuseppe D'Angelo. *A proposal for a type trait to detect narrowing conversions*. URL: <https://wg21.link/p0870>

[P1818]

Lawrence Crowl. *Narrowing and Widening Conversions*. URL: <https://wg21.link/P1818>

[Ryu]

Ryu algorithm. URL: <https://github.com/ulfjack/ryu>

[WG14-N2601]

Annex X (normative): IEC 60559 interchange and extended types. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2601.pdf>